

Theory note S0 — Why the BER of BPSK in AWGN is $Q\left(\sqrt{2E_b/N_0}\right)$

Chang Young Ham

September 2026 (accompanies `src/s0_bpsk_awgn_ber.py`)

Purpose. S0 is the calibration stage of the repository. S1 and S2 reuse its signal model ($r = s + n$, threshold at zero), so the simulator is first required to reproduce a result whose exact answer is known. This note derives that answer, maps every symbol to a line of the code, and derives the statistical test the script uses to decide whether simulation and theory agree. Mathematics used: Gaussian random variables, conditional probability, the binomial estimator.

1 Signal model

One bit $b \in \{0, 1\}$ is transmitted per symbol. Binary phase-shift keying maps it to

$$s = \begin{cases} +\sqrt{E_b} & b = 1 \\ -\sqrt{E_b} & b = 0 \end{cases}$$

so that every symbol carries energy $s^2 = E_b$ (energy per bit). The channel adds white Gaussian noise. After the receiver's matched filter the decision variable is a single real number

$$r = s + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Where $\sigma^2 = N_0/2$ comes from. N_0 is the *one-sided* noise power spectral density, so the two-sided density is $N_0/2$. Passing white noise through a filter with impulse response $h(t)$ gives an output variance

$$\sigma^2 = \int_{-\infty}^{\infty} \frac{N_0}{2} |H(f)|^2 df = \frac{N_0}{2} \int_{-\infty}^{\infty} |h(t)|^2 dt = \frac{N_0}{2}$$

for a unit-energy matched filter (the middle step is Parseval's relation). Using $\sigma^2 = N_0$ instead shifts the whole simulated curve by exactly 3 dB.

2 The decision rule and the error event

Both symbols are equally likely, and the noise is symmetric, so the maximum-likelihood decision is a threshold at zero:

$$\hat{b} = \begin{cases} 1 & r > 0 \\ 0 & r \leq 0. \end{cases}$$

Condition on $b = 1$ (so $s = +\sqrt{E_b}$). An error occurs when $r \leq 0$, i.e. when $n \leq -\sqrt{E_b}$:

$$P(\text{error} \mid b = 1) = P(n \leq -\sqrt{E_b}) = P\left(\frac{n}{\sigma} \leq -\frac{\sqrt{E_b}}{\sigma}\right) = Q\left(\frac{\sqrt{E_b}}{\sigma}\right),$$

where $Q(x) = P(Z > x)$ for $Z \sim \mathcal{N}(0, 1)$ is the Gaussian tail function and we used the symmetry $P(Z \leq -x) = P(Z \geq x)$. By the same argument $P(\text{error} \mid b = 0)$ is identical, so the average bit error rate is

$$\text{BER} = Q\left(\frac{\sqrt{E_b}}{\sigma}\right) = Q\left(\sqrt{\frac{E_b}{N_0/2}}\right) = \boxed{Q\left(\sqrt{\frac{2E_b}{N_0}}\right)}. \quad (1)$$

Numerical check. The last point of the S0 sweep is $E_b/N_0 = 9 \text{ dB} = 7.94$. With $E_b = 1$, $\sigma = \sqrt{1/15.89} = 0.2509$ and the argument of Q is $1/0.2509 = 3.986$, so $\text{BER} = Q(3.986) = 3.36 \times 10^{-5}$, the value in `results/s0_ber_results.csv`. At $E_b/N_0 = 10 \text{ dB}$, the thermal-noise condition used in S1, the same calculation gives $Q(4.47) = 3.9 \times 10^{-6}$.

3 Q and erfc

The code uses `scipy.special.erfc` because SciPy does not provide Q directly. The two are related by

$$Q(x) = \frac{1}{2} \text{erfc}\left(\frac{x}{\sqrt{2}}\right) \implies \text{BER} = \frac{1}{2} \text{erfc}\left(\sqrt{\frac{E_b}{N_0}}\right),$$

which is `ber_bpsk_theory()` in the script.

4 Correspondence with the code

The script fixes $E_b = 1$, so $N_0 = 1/(E_b/N_0)$ and $\sigma = \sqrt{N_0/2}$.

Mathematics	Line in <code>s0_bpsk_awgn_ber.py</code>
E_b/N_0 (linear) from dB	<code>ebn0_lin = db_to_linear(ebn0_db)</code>
$\sigma = \sqrt{1/(2 E_b/N_0)}$	<code>sigma = np.sqrt(1.0 / (2.0 * ebn0_lin))</code>
$b \in \{0, 1\}$	<code>bits = rng.integers(0, 2, size=...)</code>
$s = \pm 1$	<code>symbols = 2 * bits - 1</code>
$n \sim \mathcal{N}(0, \sigma^2)$	<code>noise = rng.normal(0.0, scale=sigma, ...)</code>
$r = s + n$	<code>received = symbols + noise</code>
$\hat{b} = [r > 0]$	<code>decisions = (received > 0).astype(int)</code>
count $\hat{b} \neq b$	<code>n_errors += np.sum(decisions != bits)</code>
Eq. (1)	<code>0.5 * erfc(np.sqrt(ebn0_linear))</code>
z-score, Eq. (2)	<code>(n_err - expected) / np.sqrt(expected)</code>

Note that `scale` in `rng.normal` is the standard deviation σ , not the variance σ^2 .

5 Accuracy of the Monte Carlo estimate and the pass criterion

The simulator observes K errors in n bits and reports $\widehat{\text{BER}} = K/n$. Each bit is an independent Bernoulli trial with success probability $p = \text{BER}$, so $K \sim \text{Binomial}(n, p)$ and

$$\mathbb{E}[K] = np, \quad \text{Var}[K] = np(1 - p) \approx np.$$

The *relative* standard deviation of the estimate is therefore $1/\sqrt{np} = 1/\sqrt{\mathbb{E}[K]}$.

Stopping rule. The script simulates in chunks of 10^6 bits and stops a point once at least `MIN_ERRORS = 200` errors have been seen. This bounds the relative spread by $1/\sqrt{200} \approx 7\%$ at the *hardest* point: at 9 dB the required $n \approx 200/p$ is six million bits. The easy points collect far more errors in their first chunk (79,271 at 0 dB, a spread of 0.36%), so the accuracy differs by a factor of twenty across the sweep.

Pass criterion. Because of that difference a single tolerance on the ratio $\widehat{\text{BER}}/\text{BER}$ would be either too loose at 0 dB or too tight at 9 dB. The script instead standardises each point,

$$z = \frac{K - np}{\sqrt{np}}, \quad (2)$$

which is approximately $\mathcal{N}(0, 1)$ at every point if the simulator is correct, and requires $|z| < 4$. For a correct simulator the probability of failing at a given point is $P(|Z| > 4) = 6 \times 10^{-5}$, below 10^{-3} for the whole ten-point sweep. The observed values lie between -2.11 and $+2.22$, which is the expected scatter of ten standard normal draws. A modelling error is detected easily: $\sigma^2 = N_0$ instead of $N_0/2$ gives $z > 100$ at every point.

6 Shape of the curve

For large x the Gaussian tail satisfies

$$Q(x) \approx \frac{1}{x\sqrt{2\pi}} e^{-x^2/2},$$

so with $x^2 = 2E_b/N_0$ the BER falls like e^{-E_b/N_0} : exponentially in the *linear* E_b/N_0 . Plotted on a log- y axis against linear E_b/N_0 the curve would be close to a straight line. Against E_b/N_0 in dB, as in `figures/s0_ber_curve.png`, equal steps on the x -axis are equal *ratios* of E_b/N_0 , so the curve bends downward more and more steeply (the “waterfall” shape). This exponential decay is the benchmark that partial-band jamming removes in S1, where the decay becomes only inverse-linear.